効率的な Web アセット配信の技術たち

2021/03/10 社内勉強会

Web サイトを表示するにはさまざまなアセット (HTML, JS, CSS, 画像など) を効率的にユーザーの手元に届ける必要があります。

基礎技術や工夫をそれぞれ紹介します。Web フロントエンド技術寄りです。

前提知識: ネットワーク周り

インターネット

今回の主題ではないが最低限のイメージがないと後述の技術の必要性が理解できない

- コンピューターネットワーク・電気通信
- 自律・分散で管理された多数のネットワークの相互接続
- バケツリレーのようなイメージ
- IP 網

(発表時には社内研修用の資料を参照する)

Web サイトが表示されるまで

- DNS・TCP・TLS・HTTP・HTML・サブリソース・画面描画

(発表時には社内研修用の資料を参照する)

キャッシュ・リバースプロキシ・CDN

HTTP キャッシュ

キャッシュ: 結果が同じになるとわかっているなら、以前の結果を使いまわせば良い

- サーバーの負荷軽減・高速化
- ブラウザ・プロキシ上に保存
- `Cache-Control` ヘッダー
  - fresh, stale の概念
  - `max-age`: 指定秒数、fresh とみなしてよい
  - `stale-while-revalidate`: 指定秒数、stale を使って良い (裏で更新する)
  - `immutable`: 時間経過で変化しない
- `If-None-Match`, `If-Modified-Since` ヘッダー
  - 通信の往復は発生するが、ペイロード分の通信を減らせる

ブラウザ、CDN をコントロールするもので、非常に奥が深く難しい。頑張りどころ

CDN

Content Delivery Networks

物理的・ネットワーク的にユーザーに近い位置にプロキシサーバーを置いてキャッシュすることで、素早くコンテンツを配信する

- ユーザーの RTT の改善 (100 ms オーダー → 10 ms など)
- サーバーの負荷軽減・障害耐性
- Brotli, TLS 1.3, HTTP/2, HTTP/3 などの最新対応
- ネットワーク最適化

クラウド時代になって使いやすく、当たり前のものになった

参考: Content delivery networks (CDNs)

Cache Busting

キャッシュを使わせないようにするため、URL を変更する

- 長い `max-age` や `immutable` と併用することでヒット率を上げつつ更新可能に
  - 長期的なキャッシュの指定は更新できない問題を起こさないように注意
- `index.html` が含むサブリソースの URL を変更する
  - 例: `main.js?v=202101010000` などクエリに更新日時を入れる
  - 例: `main.ae1c93.js` などファイル名にコンテンツのハッシュを入れる

URL の設定は手動でやるものではなく、対応しているツールを選択することが重要。

インフラ設定では意識する。

Service Worker, CacheStorage

サイト開発者が JavaScript でプログラム可能なブラウザ上で動くプロキシ

- サイトに訪問された際にブラウザに Service Worker をインストールできる
- Service Worker はあらゆる通信を横取り・改変可能
  - トップレベルナビゲーション (= HTML のリクエスト) 含む
- Service Worker から CacheStorage を使うことでキャッシュを柔軟に指定できる
- オフライン状態でもキャッシュから返したり、裏で先読みなども

直接扱うよりも Workbox などで高機能で設定可能なプロキシとして扱うことが多い

参考: サービスワーカー API - Web API | MDN

新しいプロトコル・コーデック

HTTP/2

- HTTP/1.1 の HTTP レベルの head-of-line blocking 問題
  - 1 つの TCP 接続上の HTTP は前のリクエストが完了 (= レスポンスを受け取り終える) しないと次のリクエストが送れない
  - ブラウザは同じドメインに対して TCP 接続を 6 本に制限している
  - → HTTP/2 では 1 つの TCP 接続の上で複数の HTTP 接続を多重化
- バイナリーでデータをやりとり
- HPACK: ヘッダー圧縮
- HTTP のセマンティクスは変わらない (参考)

現状 HTTP/2 はリクエストの 64 % ぐらいらしい (残りはほぼ HTTP/1.1)

参考: 日本語 - http2 explained

HTTP/3

- 仕様としてはまだドラフト段階
- HTTP/2 の TCP レベルの head-of-line blocking 問題
  - パケットをロストすると後続が全部詰まる
  - → UDP ベースのプロトコル QUIC に
- HTTP over QUIC = HTTP/3
- 3-way ハンドシェイクの RTT の削減
- 回線や IP などを越えた QUIC 接続のマイグレーション
- プロトコルの硬直化の回避: 将来的な改善の余地の確保
- HTTP のセマンティクスは変わらない (参考)

参考: 日本語 - HTTP/3 explained

HTTP 103, link rel=preload

ほぼ間違いなく使用される HTML のサブリソースをいかに早く送りつけるか

- ~~HTTP/2 Server Push~~ ← Chrome が導入を検証していたが廃止に向かっているらしい
- 代替 → 103 Early Hints + Link ヘッダー
  - 実験段階
  - HTTP 1xx は情報ステータス: 後続するステータスが最終ステータスになる
- `<link rel=preload>`
  - HTTP 103 と比較して、サーバーの特別な対応はいらない一方で、HTML の生成が重い場合に先行してレスポンスしたい場合には使えない

参考: Chrome の HTTP/2 サーバプッシュサポート廃止検討と、103 Early Hints について - ASnoKaze blog

gzip, Brotli, WebP, HEVC

コンテンツをより小さくエンコードする

- HTTP ボディ (テキスト) の圧縮 → gzip, Brotli
  - **CDN が** 圧縮機能を提供していることがある
- マルチメディア系ファイルの圧縮
  - ラスター画像 → PNG, GIF, WebP, AVIF
  - ベクター画像 → SVG
  - ビデオ → 要調査 (H.264, WebM, HEVC, AV1 など？)
- 圧縮パラメーターの調整: PNG の色数など
- Zopfli, Mozjpeg などのフォーマット互換の高性能代替

アイコンなら PNG ではなく SVG、アニメーションには GIF ではなく MP4 など、
コンテンツに合わせて適切なメディアを選択する必要がある

新しい技術はクライアントの環境によって使えないことがある

ビルド・不要な部分の削除

バンドル

多数のモジュールを読み込むため、1 つにまとめる

- 消費する接続数を減らす
  - HTTP/1.1 では 1 HTTP = 1 TCP & 同時 6 接続まで
  - まだまだ HTTP/1.1 は残っている (参考)
  - HTTP/2 では 1 TCP の上に HTTP が多重化される
- 主に JavaScript モジュールだが、CSS や HTML テンプレートなども含む
- バンドルしないと原理的に import の waterfall 問題が発生する
  - これは `<link rel=preload>` で回避できるはず
  - 要: 参考資料
- 各モジュールの個別のキャッシュができなくなる

Minify, Mangle, Tree Shaking, Dead Code Elimination

JavaScript のビルド時に使っていない部分を削除して全体サイズを減らす

- Minify: 動作が変わらない範囲で空白を削ったり書き方を変えたりして小さくする
- Magling: Minify の処理のひとつ・変数名などを短いものに置き換える
  - 結果的にコードが読みづらくなる
- Tree Shaking: 実際に使用するコードのみをビルド結果に含める
  - ソースコード上には存在するが呼び出されていない関数などが消える
- Dead Code Elimination: アプリ開発者の観点では「= Tree Shaking」と考えて良い

このような処理があるため、本番環境の JavaScript は読みづらいものになっている

参考: Tree-shaking versus dead code elimination | by Rich Harris | Medium

WebFont のサブセット化

表示する文字のグリフデータのみをダウンロードする

- Unicode 範囲によって分割することで、必要な範囲のみ読み込む
- CSS で Unicode 範囲を指定でき、必要に応じてブラウザがダウンロード
- (社内にノウハウがあるため、発表時には参加者に教えてもらう)

ほぼ ASCII 文字事足りる英語と違い、日本語は文字数が異常に多いので
パフォーマンス観点では日本語の WebFont はかなりの茨の道

参考: Reduce WebFont Size

環境に合わせた読み込み

レスポンシブ画像

表示する画面サイズに合わせた画像だけを読み込む

- スマホとデスクトップのディスプレイでは大きく違う物理サイズ
- アートディレクションの問題
  - スマホで小さすぎて見えない vs デスクトップで大きすぎる
  - 表示時の物理サイズに適した画像を選択したい
  - 解決策: `<picture>` + `<source media=…>`
- 解像度切り替えの問題
  - スマホで不要な高解像度 vs デスクトップで荒すぎる低解像度
  - なるべく通信量を減らしたい
  - 解決策: `<img sizes=… srcset=…>`

参考: レスポンシブ画像 - ウェブ開発を学ぶ | MDN

Differential Loading

古いブラウザ向けに JS の補間コードを使うが、新しいブラウザには読み込ませない

- 古いブラウザ向け補間コードは、新しいブラウザにはオーバーヘッドになる
- ES Modules をサポートしているかどうかで分岐して読み込ませる
  - `<script type=module>` + `<script nomodule>` で安全に分岐可能
  - ES Modules を対応している環境は ES2015 を対応していると想定できる
- 補間コードが含まれている・いない JS ファイルをそれぞれ作る必要がある
  - 1 つのソースコードを 2 回ビルドする

IE をサポートしなければ、常に ES Modules 対応環境を相手にできるので関係ない

参考: Modern Script Loading

※ `Differential Loading`: わかりやすい名前がないので Angular の命名 を拝借

必要なものを先に、そうでないものを後に

Lazy Loading, Code Splitting, Granular Chunking

JS を適切に分割し、一部は必要になるまで読み込まない

- JS サイズは初期表示速度に大きく影響する → いかに小さくするか
  - SPA でまだ表示していない別ページに関するモジュールなどは不必要
  - WYSIWYG エディタなど重い部分は後にしたい
  - 「バンドル」の必要性もあるので無闇に細かく分割すればいいわけではない
- ES2020 の dynamic import や類似の機能を使って読み込みをコントロールする
- チャンク: 概ね複数の ES モジュールをバンドルしたファイル (webpack 用語)

チャンクの境界をどこにするか、適切に読み込む実装など、非常に難しい問題なため
ビルドツールに任せる・ビルドツールの設定方法をよく知ることが重要

参考: Lazy Loading | webpack
参考: Improved Next.js and Gatsby page load performance with granular chunking

Lazy loading images, Intersection Observer

スクロールして画面内に入るまで読み込まない

- `<img loading=lazy>`: 画面内に入るまで画像を読み込ませない
- Intersection Observer:「画面内に入ったか」を JS で取得するための API
  - 最初は読み込まず画面内に入ったときだけ読み込む、などの制御ができる
  - 画像だけでなく、Element 一般で使える
- レイアウト情報は先に必要なので `<img width=… height=…>` などを設定する
  - レイアウト情報がないと読み込み時に Layout Shift が発生する
  - 画像と同じ考え方で遅延用見込み要素にはプレイスホルダーを用意する

参考: Lazy-loading images
参考: 画像による Layout Shift が無くなる Web がやって来る - mizdra's blog

script の defer, async

スクリプトの評価の前に HTML の表示を先にする

- `<script async>`: いつ動作するか保証はなく、フェッチ完了した時点でパースに割り込んで評価
- `<script defer>`: フェッチのみ先行し、パース完了後評価
- `<script async defer>`: `<script async>` と同等になる

すべての環境で使え、パフォーマンス的、安定性の良さ `<script defer>` がベスト。

参考: Efficiently load JavaScript with defer and async

Critial CSS, Internal CSS

読み込み直後の画面上に現れない部分の CSS は後で読み込む

- CSS はレンダリングをブロックする → いかに小さくするか
  - 読み込み直後はページの最上部だけが表示される
  - 画面外の要素のスタイルが数秒遅れても問題ない
- ページの最上部に関わる CSS のみ抽出し読み込み、それ以外を後にする
  - CSS の遅延読み込みの標準の方法はない → loadCSS などを使う
- `<link rel=stylesheet>` による External CSS だと往復遅延が発生するので、`<style>` による Internal CSS で HTML に埋め込む
  - TCP slow-start のため HTML (gzipped) の大きさを 14 KB に抑えるべき (参考)
  - `<style>` での埋め込みは Inline CSS と呼ばれることも (`style` 属性とは別)

参考: Defer non-critical CSS

SSR, SSG, Hydration

JS による HTML 組み立て処理を早い段階でやっておく

- JS による UI 描画完了が体感できるレベルで時間がかかる → 減らしたい
- Server Side Rendering: リクエストを受け付けたときにサーバー上でやっておく
  - ローエンドモバイル端末よりサーバーの方がスペックが良い
  - HTML があれば操作はできなくても表示はしておける
  - Hydration: HTML にデータを埋め込むなどして、ブラウザ上のライブラリの状態を「HTML 組み立てが完了した」にして、サーバー上での状態と同期する
- Static Site Generating: HTML 組み立てをビルド時にやっておく

Next.js のように SSR, SSG がシームレスに提供されていると、インタラクティブな UI ライブラリが ~~ほぼ無料で~~ 低コストに 性能改善できる (参考: 部分評価)

Node.js サーバー、SPA なのに複数の HTML ファイルへのルーティングなど
インフラ、バック、フロントの境界を越えた連携と設計が必要

計測する、立ち位置を知る

Web Vitals, Performance API

JS でユーザーの実環境のデータを計測する

- Web Vitals: Google の UX 評価指標
  - わかりやすく、Google のツールで広く使われるので手を出しやすい
  - ライブラリ でデータをサクッと取得できる
  - Core Web Vitals は 3 つの値 → ゲーミフィケーションされた環境がすでにある
  - PageSpeed Insights で簡単に確認できる
- JS からアクセスできる Performance Timeline API, Navigation Timing API, User Timing API, Resource Timing API などで細かくデータを収集できる
- データの送信は `fetch` や `navigator.sendBeacon` など

参考: パフォーマンスの測定 - ウェブ開発を学ぶ | MDN

Web Almanac, CrUX

巨大なデータを使って全体像を知る

- HTTP Archive: Web がどう構築・配信されているか記録し提供するプロジェクト
  - ものすごい量のデータを集めて記録しているらしい (クレイジー…)
- Web Almanac: HTTP Archive のデータを年単位でまとめ、専門家がデータに対するコメントを加えてわかりやすくしたレポート
  - 自力では巨大なデータを読み解けないのでこのような「まとめ」を読む
  - 「初期表示 2 sec が速いのか、遅いのか」といったことが客観できるように
- Chrome UX Report (CrUX): 実際の Chrome の使用者から集めたデータ

参考: 方法論 | HTTP Archive による Web Almanac

必要なくなくった技術

CSS スプライト

アイコンなどの小さな画像ファイルを 1 つにまとめて接続数を減らす → 不要に

- HTTP/2 の普及により HTTP 接続数を気にしなくなった
- webpack などのバンドルツールが Data URL で配布する
- SVG の利用

Domain Sharding

各サブリソースのドメインを別にして、オリジン単位の接続数制限を回避 → 不要に

- HTTP/2 の普及により HTTP 接続数を気にしなくなった
- 逆にオーバーヘッドが大きくなる

参考: Domain sharding (ドメインシャーディング) - MDN Web Docs 用語集: ウェブ関連用語の定義 | MDN

✅ 以上、思いつく限り紹介しました